

EBOOKCHATBOT

Docker Setup & Operations Guide

Development & Production Reference

DEV localhost:5173	PROD chatbot.symphonytech.com
------------------------------	---

Repository Structure

The monorepo is organised so each service owns its build definition, and shared infrastructure config lives in a dedicated folder.

```
EBOOKCHATBOT/
├── .dockerignore          # keeps build context lean
├── .gitignore
├── docker-compose.dev.yml # dev stack
├── docker-compose.prod.yml # prod stack
├── Makefile               # convenience commands
├── backend/
│   ├── Dockerfile         # multi-stage: dev + prod targets
│   ├── .env               # secrets – never committed
│   ├── .env.example       # template safe to commit
│   ├── requirements.txt
│   └── app/
│       └── main.py
├── ai/
│   └── rag_pipeline.py    # RAG pipeline – imported by backend
├── data/                  # persisted on host, mounted as volume
│   ├── ebooks/
│   └── vectorstore/
│       └── chroma.sqlite3
├── frontend/
│   ├── Dockerfile         # multi-stage: dev + builder + prod
│   ├── vite.config.js     # /api proxy for dev
│   └── src/
│       └── App.jsx
├── docker/
│   └── nginx/
│       ├── nginx.conf     # container Nginx (prod)
│       └── vps-hestia.nginx.conf # VPS host Nginx reference
└── scripts/
    ├── dev.sh
    └── prod.sh
```

Changes Made

The following modifications were made to the original codebase to support Docker-based deployment.

backend/app/main.py

The `sys.path` manipulation block at the top of the file was removed:

```
# REMOVED — these 4 lines are gone:
import sys
from pathlib import Path
ROOT_DIR = Path(__file__).resolve().parent.parent.parent
if str(ROOT_DIR) not in sys.path: sys.path.append(str(ROOT_DIR))
```

The import from `ai/` now resolves via `ENV PYTHONPATH=/app` set in the Dockerfile and docker-compose files. This is the correct container-native approach — no application code should manipulate `sys.path`.

frontend/src/App.jsx

The `VITE_BACKEND_URL` environment variable and hardcoded localhost fallback were replaced with a relative `/api` path:

```
// BEFORE
const backendUrl = import.meta.env.VITE_BACKEND_URL || "http://localhost:8000";
const response = await axios.post(`${backendUrl}/chat`, { ... })

// AFTER
const response = await axios.post(`/api/chat`, { ... })
```

The `/api` prefix is transparently stripped before reaching FastAPI by the Vite proxy in dev and by container Nginx in prod, so FastAPI routes remain unchanged.

frontend/vite.config.js

Three additions were made to the existing config:

- `server.host: "0.0.0.0"` — required for the dev server to be reachable from outside the container
- `server.port: 5173` — explicit, matching the docker-compose port mapping
- `server.proxy` — forwards `/api/*` to `http://backend:8000`, strips the `/api` prefix, and returns a clean 503 JSON response if the backend is unreachable instead of a raw `ECONNREFUSED` error

New Files Added

File	Purpose
backend/Dockerfile	Multi-stage: base → dev (uvicorn --reload) → prod (uvicorn --workers 2)
frontend/Dockerfile	Multi-stage: base → dev (Vite) → builder (npm run build) → prod (Nginx)

<code>docker/nginx/nginx.conf</code>	Container Nginx: serves Vite build, proxies /api, 900s timeouts, real_ip forwarding
<code>docker/nginx/vps-hestia/nginx.conf</code>	VPS host Nginx reference with proxy_pass http://127.0.0.1:8088
<code>docker-compose.dev.yml</code>	Dev stack with source volumes, hot-reload, healthchecks, port 5173
<code>docker-compose.prod.yml</code>	Prod stack detached, port 8088, named volume, backend internal-only
<code>Makefile</code>	make dev / prod / down-dev / down-prod / logs-dev / logs-prod / clean
<code>.dockerignore</code>	Excludes node_modules, venv, __pycache__, .env, dist from build context
<code>backend/.env.example</code>	Template for GEMINI_API_KEY, VECTOR_DB_PATH, TOP_K
<code>scripts/dev.sh</code>	Wrapper: cd to repo root and run docker compose dev
<code>scripts/prod.sh</code>	Wrapper: cd to repo root and run docker compose prod -d

SECTION 1 — DEVELOPMENT

Development Environment

Architecture

In development, both services run with live source mounts. No rebuild is needed for code changes.



Prerequisites

- Docker Desktop (or Docker Engine + Compose plugin) installed and running
- backend/.env file created from backend/.env.example with real values
- data/vectorstore/chroma.sqlite3 present on the host (your existing vectorstore)

First-Time Setup

1. Create the .env file

```
cp backend/.env.example backend/.env
# then edit backend/.env and set your GEMINI_API_KEY
```

2. Start the stack

```
# Using make (recommended)
make dev

# Or directly
docker compose -f docker-compose.dev.yml up --build
```

NOTE: The first build downloads the Python base image and installs all pip deps including torch and sentence-transformers. This can take 10-20 minutes. Subsequent starts use Docker layer cache and are much faster.

Running the Dev Stack

Start (with rebuild)

```
make dev
```

Start (no rebuild, faster)

```
docker compose -f docker-compose.dev.yml up
```

Start in background

```
docker compose -f docker-compose.dev.yml up -d
```

Stop

```
make down-dev
```

Accessing the App

Service	URL
Frontend (React / Vite)	http://localhost:5173
Backend (FastAPI)	http://localhost:8000
FastAPI docs (Swagger)	http://localhost:8000/docs
FastAPI redoc	http://localhost:8000/redoc
Health check	http://localhost:8000/

Healthchecks

Both containers have healthchecks configured. The frontend container will not start until the backend passes its health check — important because torch and ChromaDB take 30-60 seconds to load.

Container	Check	Interval	Start Period
backend	GET http://localhost:8000/	30s	60s
frontend	GET http://localhost:5173/	15s	15s

To watch health status in real time:

```
docker compose -f docker-compose.dev.yml ps
```

Hot Reload

- Backend — uvicorn runs with `--reload` watching `/app/backend` and `/app/ai`. Any `.py` file save triggers an automatic reload. No container restart needed.
- Frontend — Vite HMR is active. Any change to `frontend/src/` reflects in the browser instantly without a full page reload.

- Data — changes to data/vectorstore/ on the host are immediately visible inside the container since data/ is bind-mounted.

Debugging

View live logs (all services)

```
make logs-dev
```

View logs for a specific service

```
docker compose -f docker-compose.dev.yml logs -f backend
docker compose -f docker-compose.dev.yml logs -f frontend
```

Shell into a running container

```
docker compose -f docker-compose.dev.yml exec backend bash
docker compose -f docker-compose.dev.yml exec frontend sh
```

Inspect environment variables inside container

```
docker compose -f docker-compose.dev.yml exec backend env | grep -E
"GEMINI|VECTOR|TOP_K|PYTHON"
```

Test the backend directly (bypassing Vite proxy)

```
curl http://localhost:8000/
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "what is injection molding", "mode": "short"}'
```

Test the /api proxy (through Vite)

```
curl -X POST http://localhost:5173/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "test", "mode": "short"}'
```

Common Issues — Dev

Symptom	Fix
Frontend starts but shows "Backend unavailable"	Backend is still loading model. Wait for healthcheck to pass. Check: make logs-dev
Vite proxy returns 503 JSON	Expected fallback — backend not ready yet or crashed. Check backend logs.
backend container keeps restarting	Usually a Python import error or missing .env key. Run: docker compose -f docker-compose.dev.yml logs backend
Changes to .py files not reloading	Confirm the file is inside backend/ or ai/ — uvicorn only watches those dirs

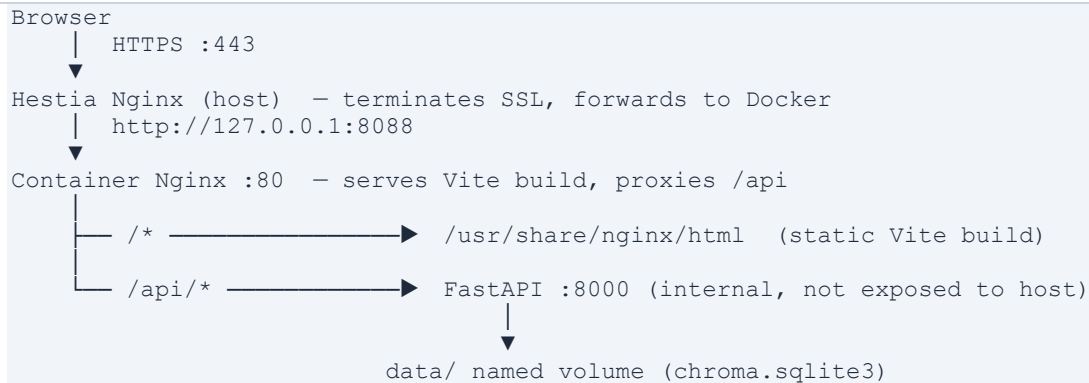
Port 5173 or 8000 already in use	Another process is using the port. Run: <code>lsof -i :5173</code> or <code>lsof -i :8000</code> and kill it
node_modules or dist inside container is stale	Rebuild: <code>docker compose -f docker-compose.dev.yml build frontend</code>

SECTION 2 — PRODUCTION

Production Environment

Architecture

In production there are no source mounts. Images are built and frozen. SSL is terminated by the Hestia-managed host Nginx; the Docker stack is internal-only on port 8088.



Prerequisites

- VPS with Docker + Compose installed
- Hestia control panel managing host Nginx for chatbot.symphonytech.com
- backend/.env on the VPS with production values
- data/ directory on the VPS with the populated vectorstore
- Port 8088 firewalled from public access (only localhost should reach it)

VPS Nginx Setup (Hestia)

Only one value needs changing in the Hestia panel. Everything else in the host Nginx config is untouched.

In the Hestia panel

- Navigate to: Web → chatbot.symphonytech.com → Edit
- Set the Proxy / Reverse Proxy URL to:

```
http://127.0.0.1:8088
```

Firewall — block port 8088 from public

```
sudo ufw deny 8088
sudo ufw allow 80,443
sudo ufw status
```

NOTE: Port 8088 must NOT be publicly reachable. All traffic should arrive via the Hestia Nginx proxy on 443. The container Nginx config trusts X-Forwarded-For from 127.0.0.1 only, so this is both a security and a correctness requirement.

Deploying

First deploy

```
# On the VPS, from repo root:
cp backend/.env.example backend/.env
# Edit backend/.env with production values
nano backend/.env

make prod
# or: docker compose -f docker-compose.prod.yml up --build -d
```

Redeploy after code changes

```
git pull
make prod
# Compose rebuilds only changed layers, then restarts affected containers
```

Stop the stack

```
make down-prod
```

Accessing the App

Service	URL
Application (public)	<code>https://chatbot.symphonytech.com</code>
Container Nginx (internal)	<code>http://127.0.0.1:8088</code> (VPS localhost only)
FastAPI (internal)	Not exposed to host — internal Docker network only

Healthchecks

Healthchecks mirror dev but the frontend container waits for backend to be healthy before Nginx starts — preventing 502 errors on cold start while the model loads.

Container	Check	Interval	Start Period
backend	<code>GET http://localhost:8000/</code>	30s	60s (model load time)
frontend	<code>GET http://localhost:80/</code>	20s	10s

Check container health status:

```
docker compose -f docker-compose.prod.yml ps
```

Data Persistence

The vectorstore is mounted as a named volume bound to `./data` on the host. This means:

- The SQLite database survives container restarts and rebuilds
- `docker compose down` does NOT delete the data (only `docker compose down -v` would)
- To update the vectorstore, copy new files into `./data/vectorstore/` on the host — no container rebuild needed

WARNING: Never run `docker compose down -v` in production. The `-v` flag deletes named volumes including the vectorstore.

Debugging — Production

View live logs

```
make logs-prod
```

View logs for a specific service

```
docker compose -f docker-compose.prod.yml logs -f backend
docker compose -f docker-compose.prod.yml logs -f frontend
```

Check container health and status

```
docker compose -f docker-compose.prod.yml ps
```

Shell into a running container

```
docker compose -f docker-compose.prod.yml exec backend bash
docker compose -f docker-compose.prod.yml exec frontend sh
```

Test backend from inside VPS (bypassing Nginx)

```
# Direct to container Nginx
curl http://127.0.0.1:8088/

# Test /api proxy through container Nginx
curl -X POST http://127.0.0.1:8088/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "test", "mode": "short"}'
```

Test Nginx config inside container

```
docker compose -f docker-compose.prod.yml exec frontend nginx -t
```

Reload Nginx config without downtime

```
docker compose -f docker-compose.prod.yml exec frontend nginx -s reload
```

Check real client IP is forwarding correctly

```
# Should show visitor IP, not 127.0.0.1 or Docker internal IP
```

```
docker compose -f docker-compose.prod.yml logs frontend | grep "GET /"
```

Common Issues — Production

Symptom	Fix
502 Bad Gateway from public URL	Container Nginx is up but backend is not healthy yet. Wait ~60s for model load. Check: <code>make logs-prod</code>
503 from /api/chat	Backend container is down or unhealthy. Check: <code>docker compose -f docker-compose.prod.yml ps</code>
Site shows old version after redeploy	Browser cached old assets. Hard refresh (Ctrl+Shift+R). Static assets have 1-year cache headers.
Backend container exits immediately	Missing or wrong <code>.env</code> value. Check: <code>docker compose -f docker-compose.prod.yml logs backend</code>
Hestia proxy returns 502	Docker stack not running on port 8088. Check: <code>docker compose -f docker-compose.prod.yml ps</code>
Real IP shows as 127.0.0.1 in logs	Confirm <code>set_real_ip_from 127.0.0.1</code> is in <code>docker/nginx/nginx.conf</code> and rebuild frontend image
Vectorstore data missing after restart	Volume mounted correctly — check <code>./data</code> exists on host with correct permissions

Makefile Quick Reference

Command	Action
make dev	Build and start dev stack (attached, with logs)
make build-dev	Build dev images only (no start)
make down-dev	Stop and remove dev containers
make logs-dev	Follow dev logs (all services)
make prod	Build and start prod stack (detached)
make build-prod	Build prod images only (no start)
make down-prod	Stop and remove prod containers
make logs-prod	Follow prod logs (all services)
make ps	Show running containers for both stacks
make clean	Prune stopped containers, dangling images, unused networks

Environment Variables

All environment variables live in backend/.env. This file is never committed to git.

Variable	Description	Example
GEMINI_API_KEY	Google Gemini API key for LLM responses	Alza...
VECTOR_DB_PATH	Path to ChromaDB SQLite file inside container	/app/data/vectorstore/chroma.sqlite3
TOP_K	Number of chunks retrieved from vectorstore per query	10

NOTE: VECTOR_DB_PATH is already set correctly for the container path in .env.example. Do not change it unless you move the data directory.